

CS4243 Cheatsheet

Emilia Ayu

Misc. Operations

Mean/Std. dev. $\bar{x} = \frac{1}{n} \sum_i x_i$, $\sigma = \sqrt{\frac{1}{n} \sum_i (x_i - \bar{x})^2}$

Normalisation $x' = \frac{x - \mu}{\sigma}$ (or min-max: $\frac{x - \min}{\max - \min}$)

Gaussian (1D) $G(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-x^2/2\sigma^2}$

Gaussian (2D) $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$

Accuracy: $\frac{TP + TN}{TP + TN + FP + FN}$ **Precision:** $\frac{TP}{TP + FP}$

Recall: $\frac{TP}{TP + FN}$ **F1-score:** $\frac{2PR}{P + R}$

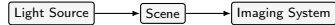
Line eqns: $y = mx + b$; $x \cos \theta + y \sin \theta = \rho$ **Euclidean dist:** $d = \sqrt{\sum_i (x_i - y_i)^2}$

Orientation: $\theta = \tan^{-1}\left(\frac{f_y}{f_x}\right)$ **Magnitude:** $\|\nabla f\| = \sqrt{f_x^2 + f_y^2}$
 $\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$ $\|\nabla f\| = \sqrt{G_x^2 + G_y^2}$

Arctan: $\arctan(0) = 0$, $\arctan(\pm 1) = \pm \frac{\pi}{4}$
 $\arctan(\pm \frac{1}{\sqrt{3}}) = \pm \frac{\pi}{6}$, $\arctan(\pm \sqrt{3}) = \pm \frac{\pi}{3}$

Lecture 1 - Computer Vision

Digital Image Acquisition



Illuminance ($i(x, y)$, $0 \leq i \leq \infty$): amount of source illumination incident on scene.

Reflectance ($r(x, y)$, $0 \leq r \leq 1$): amount of source illumination reflected by objects on scene.

Exposure: amount of time incident light can reach imaging sensor.

Aperture: how much light you let in (too much: bokeh, too narrow sharp image)

Shutter Speed: how much time you give light to accumulate (slow: motion blur, fast: freeze action)

	Spatial Resolution	Intensity Resolution
Definition	smallest discernible detail	smallest discernible δ intensity level
Measurement	pixels / unit distance	bit depth / pixel
Determined by	sampling	quantisation
Effects of low res	pixelation	false contouring

File size (bits): $b = M \cdot N \cdot k$

M : rows, N : columns, k : bits/pixel, b : total bits

File Extensions

.bmp: raw bit file (largest size)

.png: predicts pixels based on neighbours, reconstructs original pixel value (lossless)

.jpg: 8x8 image blocks represented by 64 Discrete Cosine Transform coefficients, reconstruction is approximate (lossy). Zigzag ordering captures larger coefficients first; more efficient for Huffman entropy encoding

Colour Representation

RGB Space $im(x, y, b) = y$ pixels down, x pixels to right in the b th channel

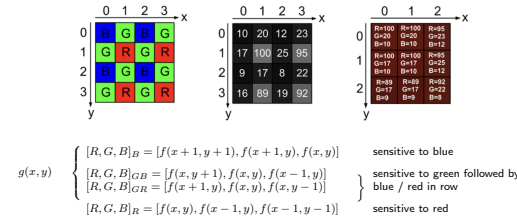
Normalized RGB Separates colour from intensity. $< \frac{R}{R+G+B} + \frac{G}{R+G+B} + \frac{B}{R+G+B}$

HSV Space: Hue = colour (0–360°, 0 = red (orange → pink); Saturation = purity (0–1, 0 = grey); Value

= brightness (0–1, 0 = black). Separates colour from intensity → robust to lighting

Bayer Filter

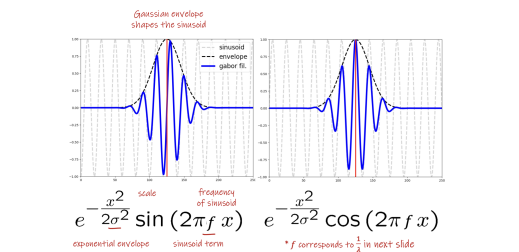
Each pixel only stores one color sample over it (RGB). Reduces intensity resolution.



Pros: Robust to outliers and noise due to local voting system. Ordinary least squares takes distance from *all points*. Can be made more efficient by voting only near its measured gradient orientation.

Lecture 4 - Regions

Filter banks: collection of multiple filter kernels, where one filter kernel responds to one local pattern. **Gabor Filter:** fix general math function with a range of parameters. Composed of sinusoids and Gaussian envelope.



f : frequency selects spacing between waves; σ : scale determines size of region covered by Gaussian envelope. **2D Gabor formula:**

$$f_{mn} = \underbrace{\frac{1}{2\pi\sigma^2} \exp\left[-\frac{m^2 + \gamma^2 n^2}{2\sigma^2}\right]}_{\text{exponential (Gaussian) envelope selects a local area}} \underbrace{\sin\left[\frac{2\pi(\cos[\omega]m + \sin[\omega]n)}{\lambda} + \phi\right]}_{\text{2D-Sinusoid selects orientation and frequency}}$$

ω – orientation in 2D; λ – wavelength of sinusoid (inversely proportional to f); ϕ – phase of sinusoid; shift from center of kernel

γ – aspect ratio of envelope coverage ($\gamma = 1$ means circular, else elliptical); σ – rate of decay of exp. envelope

Multi-Scale Reasoning

Smoothing: suppress fine detail → focus on larger structure.

Gaussian pyramid: blur, then downsample (fixed rate). Smooth first to avoid aliasing (high freq. masquerading as low).

Scale selection: filter at multiple scales; pick the response peak (filter size matching structure size).

Single vs Multi-scale Representation Given r_i, ρ_i corresponding to response and scale at i .

Single scale: $r = [r_1, r_2, \dots, r_k]$
Multi-scale: $r = [r_1^{\rho_1}, r_2^{\rho_1}, \dots, r_k^{\rho_1}, r_1^{\rho_2}, \dots, r_k^{\rho_2}]$.

Textures

Textures occur over an area. To detect, group pixels by region Ω and average filter responses over region.

1. $r(x, y) = [r_1(x, y), r_2(x, y), \dots, r_k(x, y)]$ Run k different filters to get k responses.

2. $t_k(\Omega) = \frac{1}{|\Omega|} \sum |r_k(x, y)|$. Take absolute value of every pixel's response to filter k in region Ω and sum. Divide pixels by $|\Omega|$ for average.
 $t_k(\Omega)$ is a pooled response of filter k across Ω
3. $t(\Omega) = [t_1(\Omega), t_2(\Omega), \dots, t_k(\Omega)]^T$

To compare textures, $D(\Omega_A, \Omega_B) = ||t(\Omega_A) - t(\Omega_B)||$, i.e. D is the Euclidean distance.

Classifiers

1. Nearest Neighbour Classifier: $i^* = \arg \min_i ||t(\Omega) - t(\Omega_i)||$ where $\hat{c} = y_{i^*}$
2. Nearest Class Mean: $\mu_c = \frac{1}{N_i} \sum i : y_i = ct(\Omega_c)$ where $\hat{c}(\Omega) = \arg \min ||t(\Omega) - \mu_c||$



Lecture 5 - CNNs

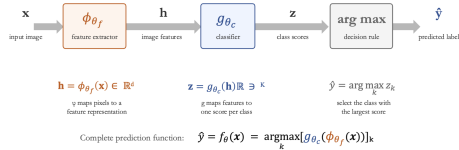
Receptive field: set of image pixels an intermediate output pixel depends on.

Recursive formula (per layer l , with kernel k_l , stride s_l):
 $r_l = r_{l-1} + (k_l - 1)j_{l-1}$, $j_l = j_{l-1} s_l$ base case:
 $r_0 = 1, j_0 = 1$

Training: Training Images → Image Features → Training with Labels → Learned Classifier

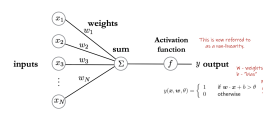
Testing: Test Image → Image Features → Learned Classifier → Prediction

Stages of Classifier



- θ_f is fixed and human-designed for traditional CV and learned for deep learning
- θ_c is learned for both traditional CV and deep learning
- $[\theta_f, \theta_c]$ are learned together ("end-to-end") for deep learning

Perceptron



#params = weights (edges) + biases (one per non-input neuron).
Neurons = all nodes except the input layer.

MLP Formalism

Input: $a^{(0)} = u$; u : input feature vector
Hidden layers:

$$a^{(\ell)} = \sigma(W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}), \ell = 1, \dots, L - 1$$

$a^{(\ell)}$: representation; $W^{(\ell)}$: weights; $b^{(\ell)}$: biases in layer ℓ ; σ : non-linear activation function

Output layer: $z = a^{(L)} = W^{(L)}a^{(L-1)} + b^{(L)}$

Convolution

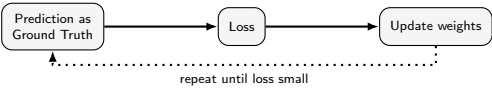
No of learnable parameters: $k^2 \cdot C_{in} \cdot C_{out} + C_{out}$

Strided. Downsample each feature map. Reduce spatial resolution. **Dilated.** Enlarge receptive field without adding params or reducing resolution.

$$H_{output} = \left\lfloor \frac{H_{input} + 2P - D(K-1) - 1}{S} \right\rfloor + 1$$
$$K_{eff} = d(k-1) + 1$$

Logits: unrestricted real-value scales → convert into probability distribution using softmax.

Loss functions



Examples

1. Localisation loss: $L_{loc} = (\hat{x} - x)^2 + (\hat{y} - y)^2$. Distance-based, naturally continuous.
2. $L_{0/1}(\hat{y}, y) = \begin{cases} 0, & \hat{y} = y \text{ (correct)} \\ 1, & \hat{y} \neq y \text{ (wrong)} \end{cases}$. Discrete
3. Cross-entropy loss: $L_{CE} = -\log(p_y)$. Continuous — softly penalizes low confidence in the correct class

Object Detection

Window-based: Build/train model → Generate candidate windows → Score candidates → Resolve detections with NMS.

Performance Evaluation Area of Overlap Measure
 $AO(B_{gt}, B_p) = \frac{|B_{gt} \cap B_p|}{|B_{gt} \cup B_p|}$

mAP (mean Average Precision): standard object-detection metric, 0–1 or 0–100%. We want high precision (few false positives) and high recall (few false negatives).

- $AP(c, \tau)$ = area under precision–recall curve (class c , $IoU \geq \tau$)
- $mAP(\tau)$ = mean AP across classes

$$AP(c, \tau) = \int_0^1 p(r) dr \quad mAP(\tau) = \frac{1}{|C|} \sum_{c \in C} AP(c, \tau)$$

CNN-Based Object Detection

R-CNNs (Region-Based ConvNets): Generate region candidates, extract CNN features for each region, regress bounding box (x,y,h,w) for each region, predict label per region.

Fast R-CNNs: Extract CNN features in one pass by

locating (x, y) in CNN feature maps and extracting features w ROI pooling.

YOLO (v2) – Anchor-based Detection

- Split image into $S \times S$ cells, each with predefined **anchors** (boxes w/ fixed sizes & aspect ratios)
- Predict at every (grid location, anchor) pair
- Objectness:** $P(obj)$ for box. Predict class + box, skip if no object.
- Class score:** $P(class | obj)$
- Box adjustment:** $P(coords | obj)$, offsets to anchor
- Loss:** $\mathcal{L} = \mathcal{L}_{obj} + \lambda_{box} \mathcal{L}_{box} + \mathcal{L}_{class}$; $\mathcal{L}_{obj}$: all anchors; $\mathcal{L}_{box}, \mathcal{L}_{class}$: object anchors only

Topic 6 - Segmentation

Separation of images into regions/objects.

Semantic: each pixel has a class label. Identify foreground and background.

Instance: Each pixel has an object it belongs to.

Panoptic: Combine semantic and instance.

k-Means Clustering: Iteratively re-assign points to nearest cluster center

- Randomly initialize the cluster centers $c_1, \dots, c_K$
- For each point p_i , find the closest c_j to put p_i in
- Set c_j to be mean of points in cluster j
- Repeat, if c_j have changed up to some threshold

- Pros: Simple, Converges to local min.
- Cons: Setting K , Sensitive to initial centers (Since k-means converges to local min.), Sensitive to outliers (Can add more clusters), Assumes spherical clusters

Simple Linear Iterative Clustering (SLIC)

- Superpixel:** Group of pixels sharing common traits.
- No of pixels: n_{tp} ; Target no of superpixels: n_{sp}
- Initial width of each superpixel: $s = \sqrt{n_{tp}/n_{sp}}$
- Features: $z = [r, g, b, x, y]$
- Color distance: $d_c = ||\langle r_j, g_j, b_j \rangle - \langle r_i, g_i, b_i \rangle||$
- Spatial distance: $d_s = ||\langle x_j, y_j \rangle - \langle x_i, y_i \rangle||$
- d_{cm} and $d_{sm} = s$ set as max. expected values of d_c and d_s respectively
- $D = \sqrt{(\frac{d_c}{d_{cm}})^2 + (\frac{d_s}{d_{sm}})^2} = \sqrt{d_c^2 + (\frac{d_s}{s})^2} c^2$

- Split img. into grid of size $s \times s$. Set cluster centers as lowest gradient position in 3×3 neighborhood from superpixel center.
- For each center, check distance to all pixels within $2s \times 2s$ neighborhood. Assign pixels to closest checked center.
- Update centers using mean; repeat until converged
- Optional: Replace superpixel region by average value to create stained glass effect

- Modification of k-Means: Not random initialization, Compute pixel's distance only to closest set of centers